Практическая работа

Загрузка Linux

Цели работы:

- изучить этапы загрузки операционных систем семейства GNU/Linux
- освоить методы анализа процесса загрузки
- получить навыки настройки параметров загрузчика, диагностики проблем и оптимизации времени старта системы.

Процесс загрузки Linux представляет собой многоэтапную последовательность, включающую инициализацию оборудования, запуск загрузчика, ядра, временной файловой системы и системы инициализации.

Этапы загрузки

1. Включение питания и инициализация оборудования

После включения питания процессор начинает выполнение кода прошивки BIOS в старых системах или UEFI в современных. Этот код выполняет проверку оборудования (POST), инициализирует устройства, а затем ищет загрузочное устройство (жёсткий диск, SSD, USB и т.д.) и передает управление загрузчику.
В BIOS загрузчик считывается из первого сектора диска (MBR). В UEFI загрузчиком является файл в специальном разделе ESP (EFI System Partition), а выбор устройства осуществляется через UEFI Boot Manager.

2. Загрузчик (Bootloader)

Наиболее распространённым загрузчиком в Linux является GRUB (Grand Unified Bootloader). Его задачи:
- предоставить пользователю меню для выбора операционной системы или версии ядра
- загрузить выбранное ядро Linux в память и передать ему параметры командной строки
- загрузить в память образ initramfs (initial RAM filesystem), который будет использован ядром на следующем этапе.

Конфигурация GRUB хранится в файле /boot/grub/grub.cfg (генерируется автоматически). Пользовательские настройки (время ожидания, пункт по умолчанию, дополнительные параметры ядра и др.) задаются в /etc/default/grub и скриптах каталога /etc/grub.d/. Для обновления конфигурации после изменений используется команда ***update-grub*** (Debian/Ubuntu) или ***grub2-mkconfig*** (Red Hat/Fedora).

3. Ядро Linux

Ядро - центральный компонент операционной системы. После загрузки оно:
- инициализирует оборудование (процессоры, память, устройства).
- распаковывает initramfs в оперативной памяти и запускает из него процесс /init.
- принимает параметры командной строки, переданные от GRUB (их можно посмотреть в файле /proc/cmdline). Параметры влияют на поведение ядра и процесс загрузки (например, quiet подавляет сообщения, splash - графическая заставка, ro - монтирование корневой файловой системы только для чтения, root=UUID=... указывает корневую файловую систему).

4. Initramfs (временная корневая файловая система)

Initramfs — небольшой образ файловой системы, содержащий минимальный набор драйверов и утилит, необходимых для монтирования настоящей корневой файловой системы. Он загружается в память вместе с ядром и выполняет следующие задачи:

- загружает драйверы для дисков, контроллеров, файловых систем
- монтирует корневую файловую систему (указанную параметром root=...)
- передаёт управление процессу инициализации, расположенному в реальной корневой файловой системе (обычно /sbin/init).

Содержимое initramfs можно просмотреть с помощью команды ***lsinitramfs***: ***lsinitramfs /boot/initrd.img-$(uname -r).***

Обновление initramfs: ***update-initramfs -u*** (Debian/Ubuntu) или ***dracut --force*** (Red Hat/Fedora).

5. Система инициализации (init)

Первым процессом, запускаемым ядром, является процесс с PID 1. В современных дистрибутивах это systemd.

Его основная задача - запустить все необходимые службы (сервисы) в порядке зависимостей, обеспечивая переход к целевому состоянию (target).

Основные цели:

- **multi-user.target** - многопользовательский режим без графики
- **graphical.target** - многопользовательский режим с графическим интерфейсом
- **rescue.target** - однопользовательский режим для восстановления.

Для анализа загрузки используется утилита systemd-analyze:

systemd-analyze time - общее время загрузки

systemd-analyze blame - время запуска каждого сервиса

systemd-analyze critical-chain - цепочка критических зависимостей.

systemd-analyze plot > boot.svg - график загрузки в формате SVG.

6. Логи загрузки

Сообщения ядра хранятся в кольцевом буфере и доступны через команду ***dmesg.***

Системный журнал **systemd (journal)** содержит логи всех служб и ядра. Просмотр логов:

- для текущей загрузки: ***journalctl -b***
- фильтрация по приоритету: ***journalctl -b -p err*** (только ошибки)
- по сервису: ***journalctl -b -u имя_сервиса.service***.

Практическое задание

Задание 1: Карта загрузки системы

Цель: понять последовательность этапов загрузки Linux, определить основные компоненты и проанализировать параметры, переданные ядру.

Задачи

1. Определить систему инициализации.

2. Изучить параметры загрузки ядра и объяснить их назначение.
3. Оценить время загрузки.
4. Посмотреть загруженные модули ядра.

Порядок выполнения

1. Определите, какая система инициализации используется (PID1):

file /sbin/init ***# обычно ссылка на systemd***
systemctl --version ***# версия systemd***
ps -p 1 -o comm= ***# имя процесса с PID 1***

2. Получите параметры командной строки, переданные ядру:

cat /proc/cmdline

Изучите вывод команды, найдите и объясните следующие параметры (если они присутствуют):

- BOOT_IMAGE=... - путь к образу ядра.
- root=... - указание корневой файловой системы (может быть задана через UUID= или /dev/...)
- ro - монтирование корневой файловой системы в режиме «только чтение» на начальном этапе
- quiet - подавление вывода сообщений ядра
- splash - отображение графической заставки
- vt.handoff=... - передача управления виртуальному терминалу.

3. Определите время загрузки системы:

systemd-analyze time # или просто ***systemd-analyze***

Визуализация графика загрузки:

systemd-analyze plot > ~/boot.svg

Примечание: Команда ***systemd-analyze plot > ~/boot.svg*** не выводит данные на экран, а сохраняет детальный график загрузки в формате SVG в файл boot.svg в вашем домашнем каталоге. Этот файл можно открыть в любом браузере или программе просмотра изображений. Для проверки создания файла используйте ***ls -l ~/boot.svg***.

4. Список загруженных модулей ядра (первые 10)

lsmod | head -10

Вопросы для отчета:

1. Перечислите этапы загрузки в правильном порядке (от включения питания до появления приглашения входа).
2. Какой процесс запускается первым после ядра? Какова его роль?
3. Сколько времени заняла загрузка вашей системы (kernel + userspace)?
4. Запишите содержимое /proc/cmdline. Что означают параметры quiet, splash, ro? Какой UUID у вашей корневой файловой системы? Зачем используется UUID вместо имени устройства (например, /dev/sda1)?
5. Какие модули ядра загружены первыми? (из вывода lsmod)

Задание 2. Исследование загрузчика GRUB

Цель: изучить конфигурацию загрузчика GRUB, научиться находить доступные ядра и параметры загрузки.

Задачи

1. Понять структуру конфигурационного файла GRUB.

2. Определить все доступные пункты меню (включая подменю).
3. Определить настройки по умолчанию.

Порядок выполнения

1. Получите список доступных ядер в системе:

ls /boot/vmlinuz* | sort -V

2. Изучите структуру файла конфигурации GRUB

Посмотрите начало файла, чтобы понять его синтаксис

head -30 /boot/grub/grub.cfg

Или просмотрите файл постранично (выход по q)

less /boot/grub/grub.cfg

Конфигурационный файл /boot/grub/grub.cfg создаётся автоматически и его не рекомендуется редактировать вручную. Содержит множество служебных строк, в том числе условные операторы (if), которые не являются пунктами меню. Строки, объявляющие пункты меню, всегда содержат слово menuentry. Основной пункт меню (обычно первая запись) не имеет отступа, а вложенные пункты внутри подменю - имеют пробелы или табуляции в начале строки.

3. Получите список всех пунктов меню GRUB:

grep "menuentry" /boot/grub/grub.cfg | grep -v "if" | grep -v "export"

Извлеките только названия пунктов меню:

grep "menuentry" /boot/grub/grub.cfg | grep -v "if" | grep -v "export" | cut -d "'" -f2

Чтобы учесть вложенные пункты (с отступами), можно использовать более универсальный шаблон:

grep -E "^[[:space:]]*menuentry" /boot/grub/grub.cfg | cut -d "'" -f2

[[:space:]]* означает любое количество пробелов или табуляций перед menuentry.

4. Узнайте версию GRUB:

grub-install --version

5. Просмотрите основные настройки загрузчика:

cat /etc/default/grub

grep GRUB_TIMEOUT /etc/default/grub # время показа меню

grep GRUB_DEFAULT /etc/default/grub # пункт по умолчанию

6. Перезагрузите систему и войдите в меню GRUB (при необходимости удерживайте Shift или нажимайте Esc). Сделайте скриншот меню загрузки. Выберите пункт «Advanced options» и изучите доступные ядра.

Вопросы для отчета:

1. Сколько различных пунктов меню (включая подменю) вы обнаружили в файле grub.cfg?
1. Сколько ядер доступно для загрузки в вашей системе? Укажите их версии.
2. Какое ядро загружается по умолчанию? Как это определено в /etc/default/grub?
3. Какой параметр в файле /etc/default/grub отвечает за время отображения меню? Какое значение установлено у вас?
4. Что произойдет, если установить GRUB_TIMEOUT=0?

Задание 3. Анализ времени загрузки

Цель: научиться определять, какие службы замедляют старт системы.

Задачи

1. Получить общее время загрузки.
2. Найти самые «медленные» сервисы.
3. Построить цепочку критических зависимостей.

Порядок выполнения

1. Определите общее время загрузки (ядро + userspace):

systemd-analyze

Получите:

2. время инициализации каждого сервиса (первые 10):

systemd-analyze blame | head -10

3. критическую цепочку загрузки (самые важные зависимости):

systemd-analyze critical-chain

4. список всех сервисов и их состояние:

systemctl list-units --type=service | head -15

Проанализируйте полученные результаты, составьте таблицу из 5 сервисов с наибольшим временем запуска. Определите, какие сервисы должны загрузиться до появления сети.

Вопросы для отчета:

1. Какой сервис грузится дольше всех? Сколько времени это занимает?
2. Что означает состояние active (running) у сервиса?
3. Какой target отвечает за графический интерфейс? Как проверить, активен ли он?
4. Как переключиться в текстовый режим без перезагрузки?

Задание 4. Исследование Initramfs

Цель: понять назначение временной корневой файловой системы (initramfs).

Задачи

1. Найти файл initramfs для текущего ядра.
2. Оценить его размер и дату создания.
3. Изучить содержимое initramfs.

Порядок выполнения

1. Получить информацию о файле **initramfs**:

ls -lh /boot/initrd.img-$(uname -r)

2. Прочитать содержимое initramfs (первые 20 строк):

lsinitramfs /boot/initrd.img-$(uname -r) | head -20

3. Найти драйверы внутри initramfs:

lsinitramfs /boot/initrd.img-$(uname -r) | grep -E "\.ko$" | head -10

4. Найти скрипты инициализации внутри initramfs:

lsinitramfs /boot/initrd.img-$(uname -r) | grep "scripts"

Вопросы для отчета:

1. Какой размер у вашего **initramfs?** (в мегабайтах)
2. Для чего нужен **initramfs**?
3. Какие драйверы (модули) присутствуют в **initramfs**? Приведите 2–3 примера.
4. Что произойдёт, если удалить **initramfs**?

Задание 5: Мониторинг процесса загрузки (логи)

Цель: научиться читать и анализировать сообщения, генерируемые при загрузке.

Задачи

1. Просмотреть логи текущей загрузки.
2. Найти сообщения ядра.
3. Обнаружить возможные ошибки.

Порядок выполнения

Получите:

1. время начала текущей загрузки:

uptime -s

who -b

2. все логи текущей загрузки (просмотр через less):

journalctl -b | less

3. только сообщения ядра:

dmesg | head -30 ***# первые 30 строк***

dmesg | grep -i "error\|warn" # поиск ошибок/предупреждений

4. только логи с ошибками (приоритет err и выше):

journalctl -b -p err

5. логи конкретного сервиса (например, network):

journalctl -b -u systemd-networkd

Найдите в логах момент монтирования корневой файловой системы (например, ***journalctl -b | grep "Mount.*root"***). Определите, какие устройства были обнаружены при загрузке (диски, сетевые карты). Проверьте, есть ли в логах предупреждения (warnings).

Вопросы для отчета:

1. В какое время началась текущая загрузка системы?
2. Были ли ошибки при загрузке? Если да, то какие?
3. Какой сервис, судя по логам, запустился первым? (после ядра)

Задание 6. Сравнение состояний системы

Цель: проанализировать состояние уже загруженной системы.

Задачи

1. Подсчитать загруженные модули ядра.
2. Посмотреть смонтированные файловые системы.
3. Изучить сетевые интерфейсы и запущенные процессы.

Порядок выполнения

Получите информацию о:

1. количестве загруженных модулей:

lsmod | wc -l

2. смонтированных файловых системах (только блочные устройства):

mount | grep "^/dev"

3. использовании дискового пространства:

df -h

4. сетевых интерфейсах:

ip addr show

5. количестве запущенных процессов:

ps -e --no-headers | wc -l

Вопросы для отчета:

1. Сколько модулей ядра загружено в системе?
2. Какая файловая система используется для корневого каталога (`/`)?
3. Какие сетевые интерфейсы активны? Есть ли у них IP-адреса?
4. Сколько процессов работает в системе? (примерно)

Задание 7. Эксперимент с уровнями выполнения (targets)

Цель: понять разницу между различными состояниями системы в **systemd.**

Задачи

1. Изучить доступные targets.
2. Узнать текущий target по умолчанию.
3. Сравнить multi-user.target и graphical.target.

Порядок выполнения

Получите:

1. список всех доступных targets:

systemctl list-unit-files --type=target

2. текущий target по умолчанию:

systemctl get-default

3. сервисы, необходимые для multi-user.target:

systemctl list-dependencies multi-user.target | head -20

4. сервисы, необходимые для graphical.target:

systemctl list-dependencies graphical.target | head -20

5. текущий активный target:

systemctl list-units --type=target --state=active

Сравните списки зависимостей для **multi-user.target** и **graphical.target**. Какие дополнительные сервисы запускаются в графическом режиме? Определите, какой сервис отвечает за менеджер входа (display manager). Например:
grep -l "display-manager" /lib/systemd/system/* или ***systemctl status display-manager***.
Обратите внимание, что переключение в multi-user.target закроет графическую сессию. Работу следует выполнять в виртуальной машине или быть готовым вернуться в графический режим через sudo ***systemctl isolate graphical.target***.

Вопросы для отчета:

1. В каком target работает ваша система сейчас?
2. Какие сервисы есть в graphical.target, но отсутствуют в multi-user.target?
3. Как переключиться в текстовый режим (multi-user) без перезагрузки? Выполните команду (с предупреждением о закрытии графических приложений):

   sudo systemctl isolate multi-user.target

Контрольные вопросы

1. Назовите основные этапы загрузки Linux.
2. В чём различие между BIOS и UEFI?
3. Какова роль GRUB в процессе загрузки?
4. Что такое initramfs и для чего он нужен?

5. Какая информация содержится в файле /proc/cmdline?
6. Как узнать, какая система инициализации используется?
7. Перечислите основные команды systemd-analyze.
8. Как посмотреть логи только текущей загрузки?
9. Что такое target в systemd? Приведите примеры.
10. Как добавить параметр ядра временно (при загрузке) и постоянно?

Дополнительное задание
Настройка пользовательского пункта меню GRUB с дополнительными параметрами ядра

Цель: научиться добавлять собственные параметры загрузки ядра, создавать отдельный пункт меню GRUB и проверять влияние этих параметров на работу системы.

Задачи:

1. Изучить текущие параметры ядра, переданные загрузчиком.
2. Создать собственный пункт меню GRUB с дополнительными параметрами ядра.
3. Обновить конфигурацию GRUB и проверить появление нового пункта.
4. Загрузиться с новым пунктом и убедиться, что параметры применились.
5. Вернуть систему в исходное состояние.

Порядок выполнения:

1. Анализ текущих параметров ядра

1.1 Посмотрите параметры, с которыми загружено текущее ядро:

cat /proc/cmdline

Запишите их в отчёт.

1.2 Посмотрите настройки GRUB по умолчанию:

cat /etc/default/grub

Найдите строки **GRUB_CMDLINE_LINUX_DEFAULT** и **GRUB_CMDLINE_LINUX**. Объясните, чем они отличаются.

2. Создание пользовательского пункта меню

2.1 Откройте файл для дополнительных записей меню:

sudo nano /etc/grub.d/40_custom

2.2 Добавьте в конец файла следующий блок (замените YOUR_NAME на свою фамилию):

```
menuentry "Custom Boot - YOUR_NAME" {
    insmod gzio
    insmod part_gpt
    insmod ext2
    search --no-floppy --fs-uuid --set=root <UUID-вашего-корневого-раздела>
    linux /boot/vmlinuz-$(uname -r) root=UUID=<UUID-вашего-корневого-раздела> ro debug systemd.log_level=debug
    initrd /boot/initrd.img-$(uname -r)
}
```

Чтобы узнать UUID корневого раздела выполните:

findmnt -n -o UUID /

Вставьте полученное значение вместо <UUID-вашего-корневого-раздела>.

Значение параметров:

- ***debug*** - включает подробный вывод отладки ядра.
- ***systemd.log_level=debug*** - повышает уровень детализации логов systemd.

Эти параметры безопасны и не нарушат загрузку.

2.3 Сохраните файл (Ctrl+O, Enter, Ctrl+X).

2.4 Обновите конфигурацию GRUB:

sudo update-grub

В выводе должно появиться сообщение о найденном пункте **Custom Boot - YOUR_NAME.**

2.5 Проверьте, что пункт появился в конфигурации:

grep "Custom Boot" /boot/grub/grub.cfg

3. Загрузка с новым пунктом и проверка

3.1 Перезагрузите систему:

sudo reboot

3.2 В меню GRUB выберите пункт «Custom Boot - YOUR_NAME» (используйте стрелки, затем Enter).

3.3 После загрузки проверьте, что параметры применились:

cat /proc/cmdline

Убедитесь, что в выводе присутствуют ***debu*** и ***systemd.log_level=debug.***

3.4 Посмотрите, как изменился вывод логов загрузки:

journalctl -b | head -30

Сравните с предыдущей загрузкой (можно посмотреть ***journalctl -b -1***). Запишите различия в отчёт.

3.5 Проверьте общее время загрузки:

systemd-analyze time

Сравните с временем загрузки в обычном режиме. Объясните результат.

4. Продемонстрируйте преподавателю выполнение пунктов 3.1 – 3.4.

5. Возврат системы в исходное состояние

5.1 Откройте файл 40_custom снова:

sudo nano /etc/grub.d/40_custom

Удалите добавленный блок ***menuentry***.

5.2 Обновите конфигурацию GRUB:

sudo update-grub

Убедитесь, что пользовательский пункт исчез:

grep "Custom Boot" /boot/grub/grub.cf

Вопросы для отчёта

1. Чем отличаются параметры **GRUB_CMDLINE_LINUX_DEFAULT** и **GRUB_CMDLINE_LINUX** в файле /***etc/default/grub***?
2. Какие параметры ядра вы добавили в свой пункт меню и зачем?
3. Как проверить, какие параметры были переданы ядру при текущей загрузке?

4. Что изменилось в логах загрузки после добавления параметра ***debug***? Приведите примеры.
5. Повлияло ли добавление отладочных параметров на время загрузки? Если да, то как и почему?
6. Почему файл ***/boot/grub/grub.cfg*** не рекомендуется редактировать вручную? Какая команда обновляет его автоматически?
7. Что произойдёт, если в параметрах ядра допустить ошибку (например, указать несуществующий параметр)? Как это может повлиять на загрузку?